

Cloud et Réseaux Virtuels

Kubernetes

Massinissa Tighilt

03/04/2026

Table of Contents

1 Introduction et motivations

2 Rappel sur les conteneurs (Docker)

3 Introduction à Kubernetes

4 Architecture Kubernetes

5 Les objets Kubernetes fondamentaux

6 Utilisation avec kubectl

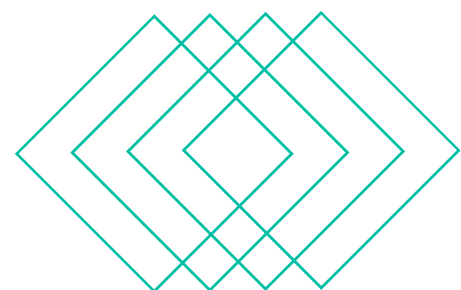


Pourquoi l'orchestration des conteneurs ?

- Les applications modernes sont souvent composées de plusieurs services :
 - API
 - base de données
 - frontend
 - services internes (cache, messaging...)
- **Chaque service est généralement exécuté dans un conteneur.**

Problème:

- Quand une application grandit, on peut avoir :
 - des dizaines ou centaines de conteneurs
 - sur plusieurs machines
- Il devient alors difficile de gérer :
 - le déploiement
 - la mise à l'échelle
 - la tolérance aux pannes





Limites de Docker seul

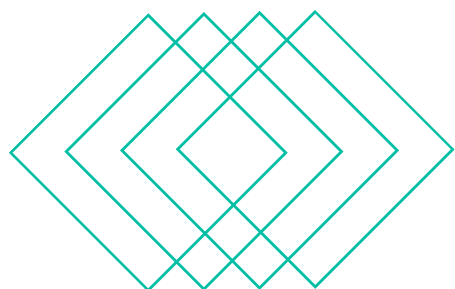
- Docker est excellent pour :
 - créer des images
 - lancer des conteneurs
 - tester localement

Mais Docker ne résout pas plusieurs problèmes en production.

- Docker ne gère pas automatiquement :
 - le redémarrage automatique sur un autre serveur
 - la répartition de charge
 - le scaling automatique
 - la gestion de clusters de machines

Exemple concret

- Si un serveur tombe :
- les conteneurs disparaissent
- l'application devient indisponible



Un orchestrateur



- Un orchestrateur de conteneurs est un système qui :
 - déploie les conteneurs
 - surveille leur état
 - redémarre ceux qui échouent
 - distribue les conteneurs sur plusieurs machines



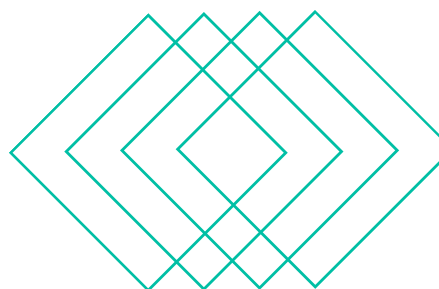
Un orchestrateur

Fonctionnalités attendues

- gérer plusieurs serveurs
- assurer la haute disponibilité
- gérer le scaling automatique
- faciliter les mises à jour sans interruption

Exemple

- Si un conteneur tombe :
 - orchestrateur → recrée automatiquement un conteneur
- Si la charge augmente :
 - orchestrateur → lance plusieurs instances



Kubernetes

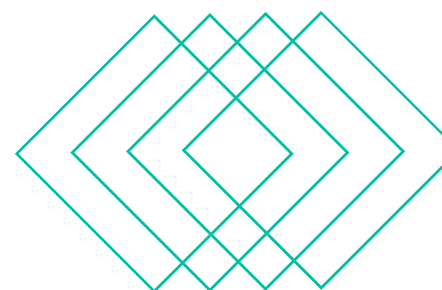
- Kubernetes est une plateforme open-source permettant de :
 - automatiser le déploiement
 - gérer des conteneurs à grande échelle
 - assurer la résilience des applications
- Il permet de gérer :
 - des applications conteneurisées
 - sur un cluster de machines

Fonctionnalités principales

- l'orchestration des conteneurs
 - le scaling automatique
 - la gestion des pannes
 - les mises à jour progressives
 - la gestion du réseau
-
- On décrit l'état souhaité :
 - **Je veux 3 instances de mon application**
 - Kubernetes s'occupe de maintenir cet état automatiquement.

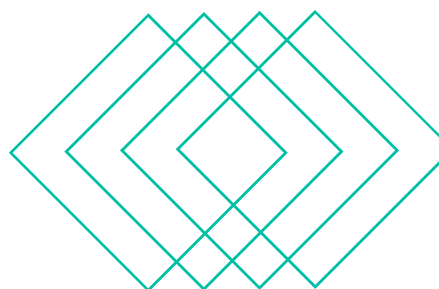


kubernetes



Kubernetes dans l'industrie

Kubernetes est aujourd'hui la plateforme standard pour le déploiement d'applications cloud.





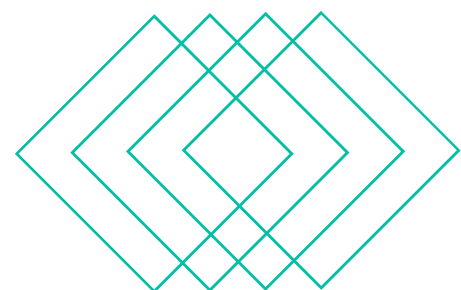
Kubernetes dans l'industrie

Pourquoi les entreprises utilisent Kubernetes ?

- gérer des milliers de conteneurs
- améliorer la résilience des systèmes
- automatiser l'infrastructure
- faciliter le déploiement continu

Écosystème Cloud

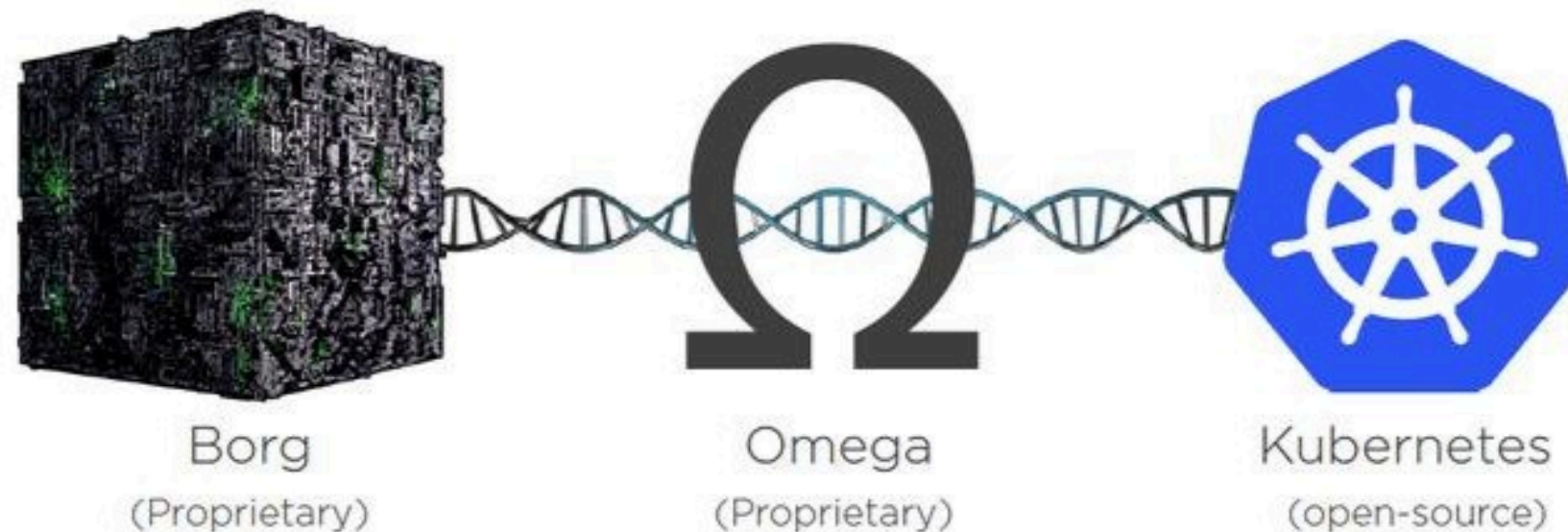
- Kubernetes est intégré dans les principaux clouds :
 - Google Kubernetes Engine (GKE)
 - Amazon Elastic Kubernetes Service (EKS)
 - Azure Kubernetes Service (AKS)



Historique : de Borg à Kubernetes

Origine chez Google

- Avant Kubernetes, Google utilisait un système interne appelé Borg.
- Borg permettait déjà de :
 - gérer des milliers de machines
 - orchestrer des conteneurs
 - assurer la haute disponibilité
- Google a ensuite développé :
 - Omega → Kubernetes
- Kubernetes reprend les idées de Borg mais sous forme open-source.



Évolution

- Google a ensuite développé :
 - **Omega → Kubernetes**
- Kubernetes reprend les idées de Borg mais sous forme open-source.
- Création
 - 2014 : publication de Kubernetes
 - projet soutenu par Google
- aujourd'hui maintenu par la Cloud Native Computing Foundation (CNCF).
- Signification de K8s
 - Kubernetes → K8s
 - Il y a 8 lettres entre K et s.

Kubernetes comme orchestrateur

Fonctionnement

- On décrit l'application dans un fichier :
 - **replicas: 3**
 - **image: nginx**
- Kubernetes :
 - lance les conteneurs
 - surveille leur état
 - recrée ceux qui tombent

Exemple

- Si un serveur tombe :
 - Kubernetes va :
 - relancer les conteneurs
 - sur une autre machine
- L'application reste disponible.

Cas d'usage typiques

Microservices

- Applications composées de plusieurs services :
 - frontend
 - API
 - base de données
 - cache
- Kubernetes gère chaque service dans des conteneurs.

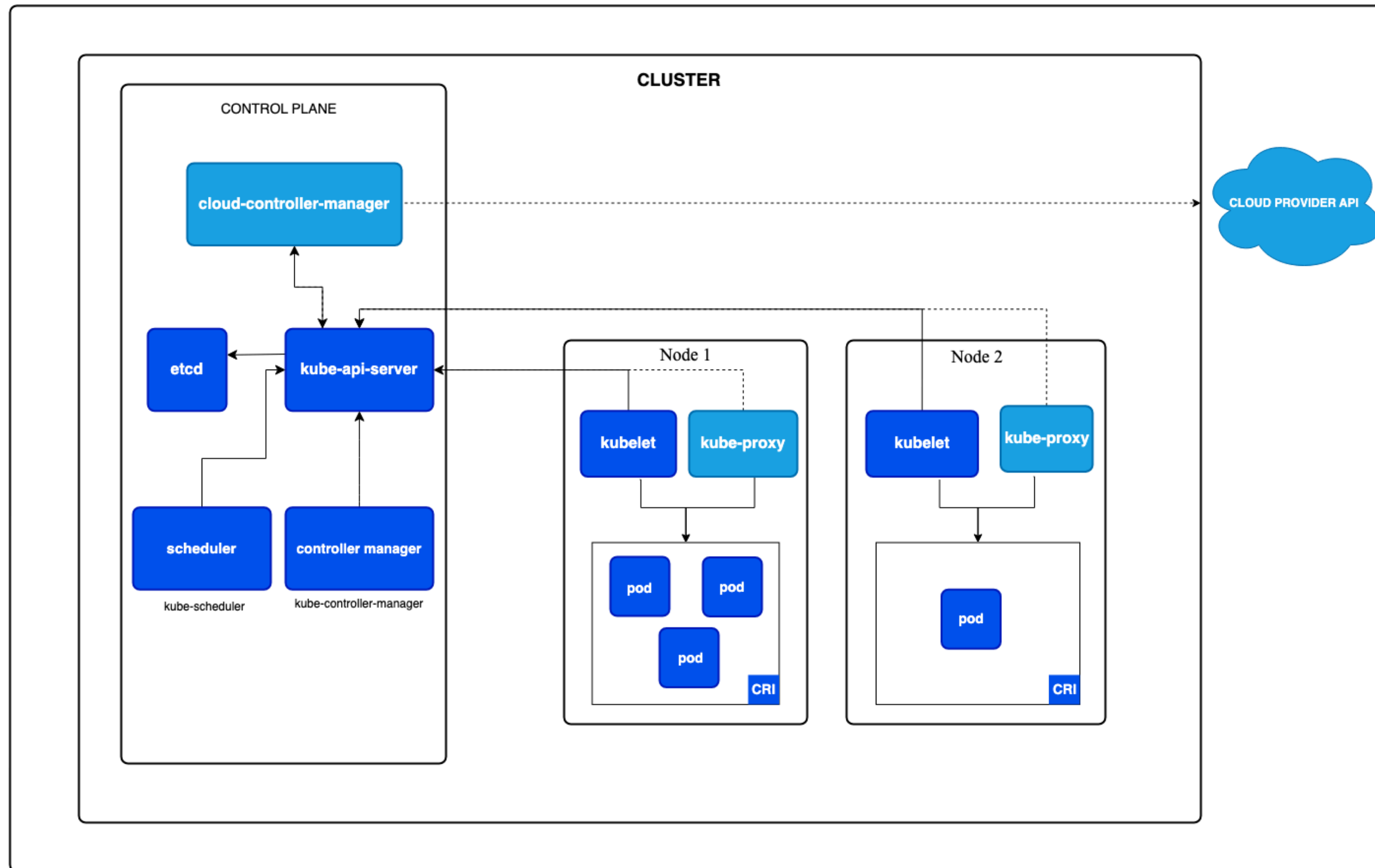
Applications cloud

- Déploiement d'applications :
 - sur plusieurs machines
 - avec montée en charge automatique

CI/CD et DevOps

- Kubernetes facilite :
 - les déploiements automatisés
 - les mises à jour sans interruption
 - les environnements reproductibles

Vue globale d'un cluster Kubernetes



Vue globale d'un cluster Kubernetes

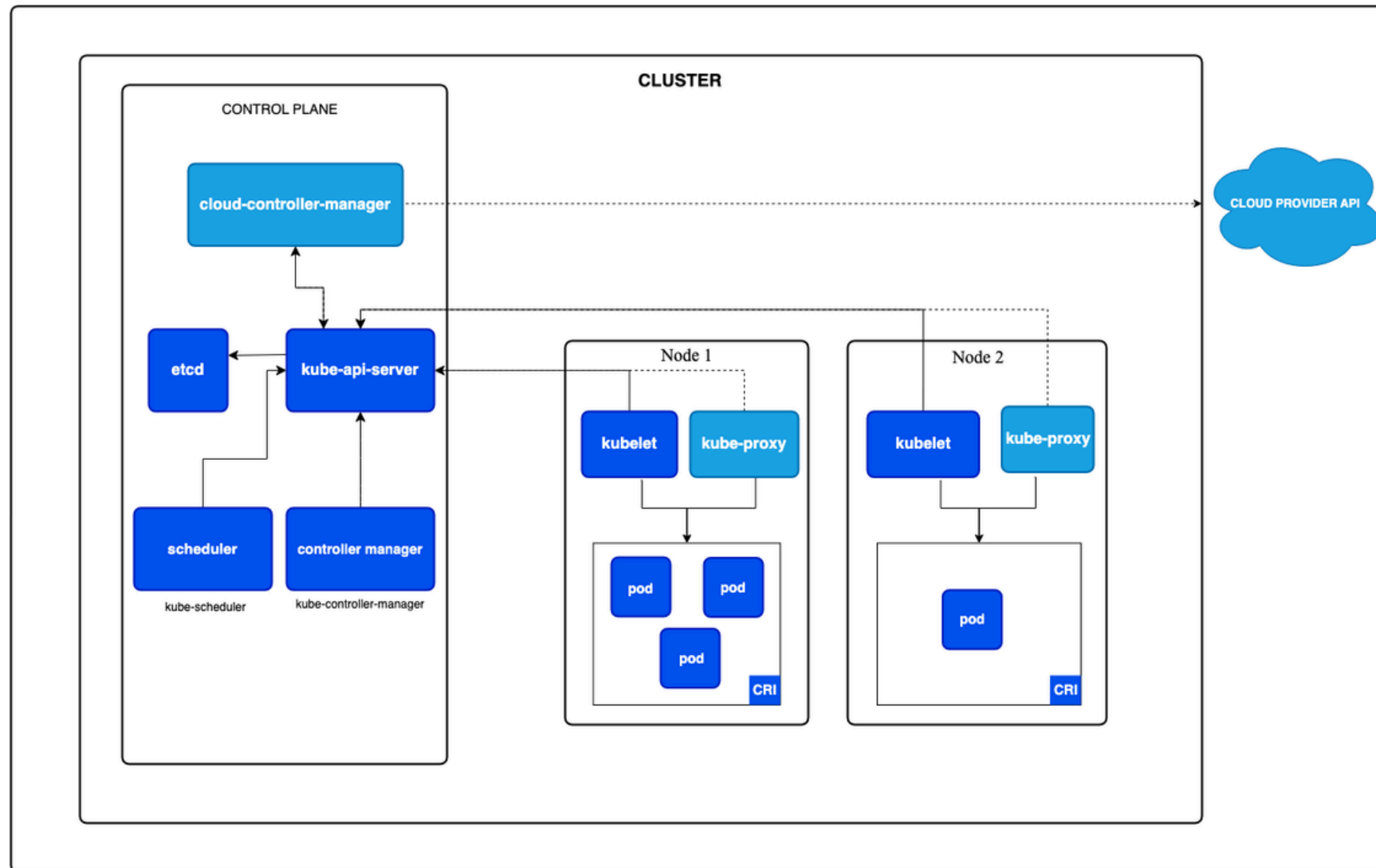
Deux types de machines

- Control Plane

- Le cerveau du cluster
- Il décide :
 - où lancer les conteneurs
 - comment gérer le cluster

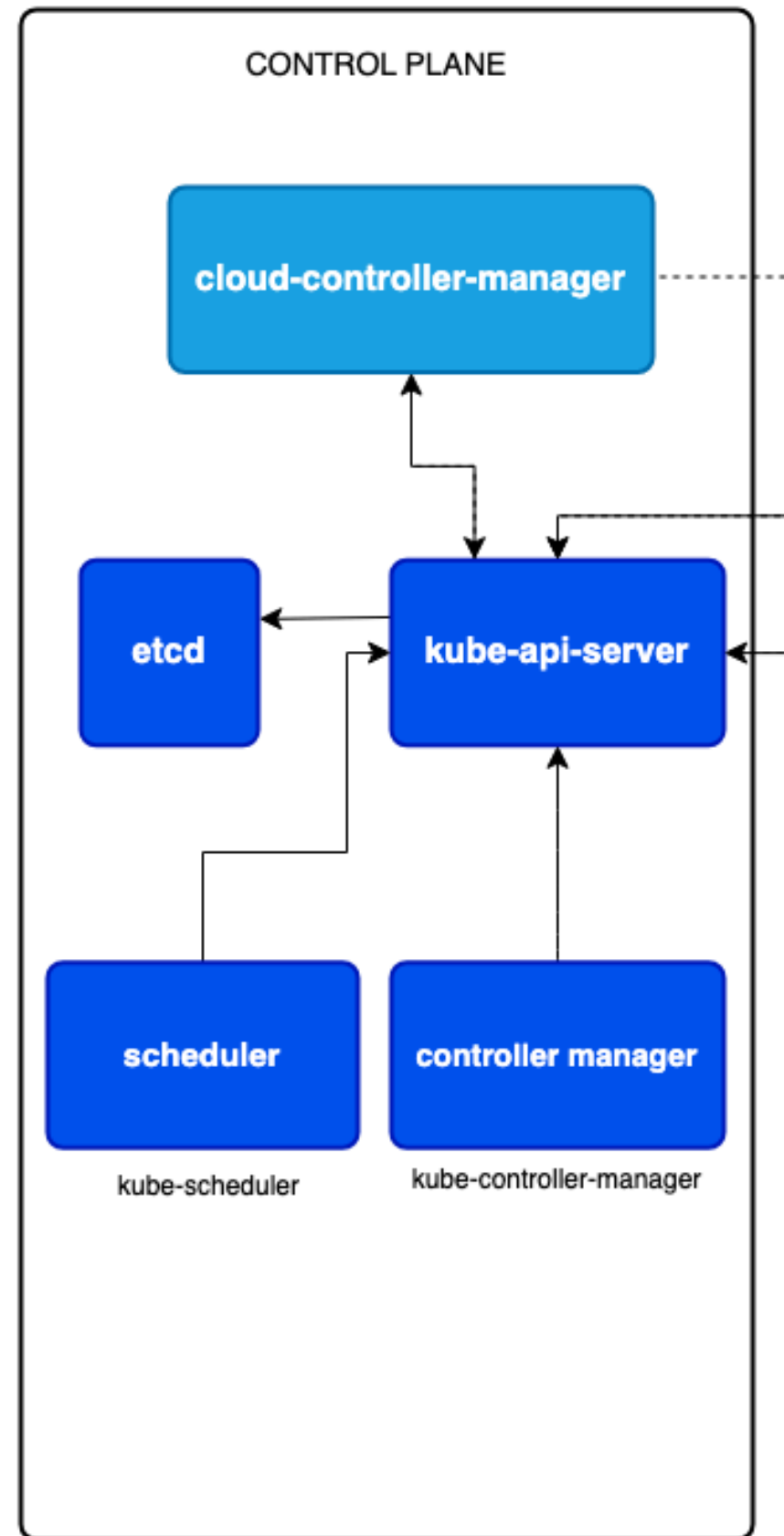
- Worker Nodes

- Les machines qui exécutent :
 - les conteneurs
 - les applications
- Chaque Node peut exécuter plusieurs Pods.



Architecture globale

Control Plane





Architecture globale

Control Plane

Rôle du Control Plane

- Le Control Plane est le cerveau du cluster Kubernetes.
- Il prend toutes les décisions concernant :
 - la gestion du cluster
 - l'état des applications
 - la planification des conteneurs

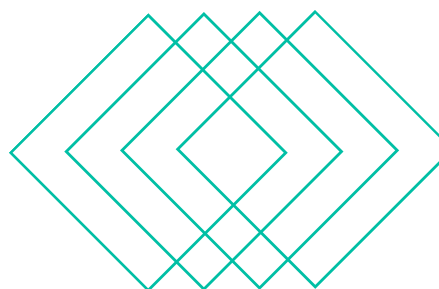
Responsabilités principales

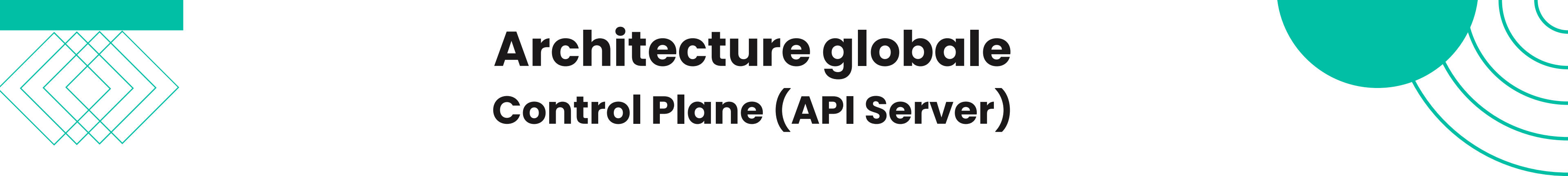
- surveille l'état du cluster
 - décide où lancer les conteneurs
 - assure la cohérence du système
- 

Composants du Control Plane

- API Server
- Scheduler
- Controller Manager
- etcd

Ces composants travaillent ensemble pour gérer le cluster.





Architecture globale

Control Plane (API Server)

Composant central

- L'API Server est le point d'entrée principal de Kubernetes.
- Toutes les interactions avec le cluster passent par lui.

Qui utilise l'API Server ?

- les utilisateurs (via kubectl)
- les applications
- les composants internes de Kubernetes

Exemple de commande

- Lorsque l'on exécute :
 - **kubectl get pods**
- la commande envoie une requête à l'API Server.
 - L'API Server :
 - reçoit la requête
 - consulte la base etcd
 - retourne la réponse

L'API Server agit comme une API REST centrale pour gérer les ressources Kubernetes.



Architecture globale

Control Plane (Scheduler)

Rôle du Scheduler

Le Scheduler est responsable de placer les Pods sur les machines du cluster.

Quand un nouveau Pod doit être créé :

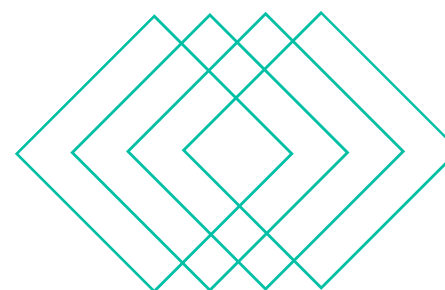
- sur quelle machine doit-il être lancé ?

Le Scheduler analyse :

- les ressources disponibles (CPU, mémoire)
- les contraintes réseau
- les règles de placement

- **Exemple**

- Si un cluster possède 3 machines :
 - Node 1 → CPU saturé
 - Node 2 → mémoire disponible
 - Node 3 → ressources libres
- Le Scheduler choisira le Node le plus adapté.
- Optimiser :
 - la performance
 - l'utilisation des ressources
 - la disponibilité





Architecture globale

Control Plane (Controller Manager)

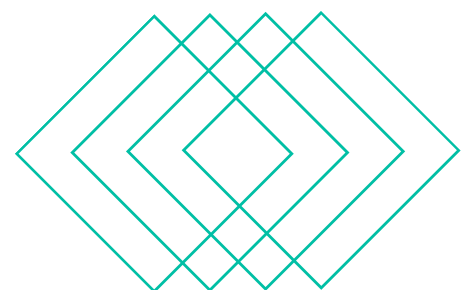
Rôle du Controller Manager

Le Controller Manager surveille en permanence l'état du cluster.

Il compare :

- l'état actuel
- l'état souhaité

- Principe fondamental de Kubernetes:
 - Kubernetes fonctionne avec un modèle déclaratif.
 - Exemple :
 - Je veux 3 Pods
- **Fonctionnement**
 - Si un Pod disparaît :
 - Etat actuel : 2 Pods
 - Etat souhaité : 3 Pods
 - Le Controller Manager :
 - **crée automatiquement un nouveau Pod.**





Architecture globale

Control Plane (Controller Manager)

Rôle du Controller Manager

Le Controller Manager surveille en permanence l'état du cluster.

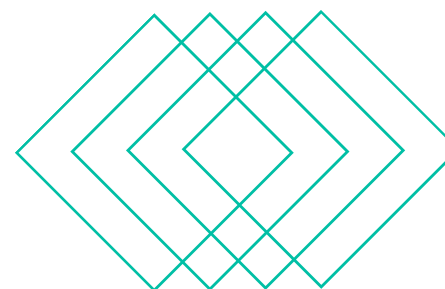
Il compare :

- l'état actuel
- l'état souhaité

Types de contrôleurs

- Replica Controller
- Node Controller
- Endpoint Controller

- Principe fondamental de Kubernetes:
 - Kubernetes fonctionne avec un modèle déclaratif.
 - Exemple :
 - Je veux 3 Pods
- **Fonctionnement**
 - Si un Pod disparaît :
 - Etat actuel : 2 Pods
 - Etat souhaité : 3 Pods
 - Le Controller Manager :
 - **crée automatiquement un nouveau Pod.**





Architecture globale

Control Plane (etcd : base de données du cluster)

Qu'est-ce que etcd ?

- etcd est une base de données distribuée utilisée par Kubernetes.
- Elle stocke toutes les informations du cluster.

Données stockées

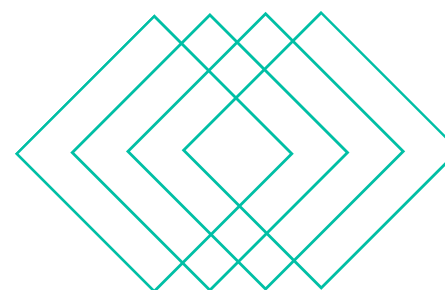
- configuration du cluster
- état des Pods
- état des Nodes
- informations réseau

Exemple

- Si on crée un Pod :
 - **kubectl apply -f pod.yaml**
- L'information est enregistrée dans etcd.

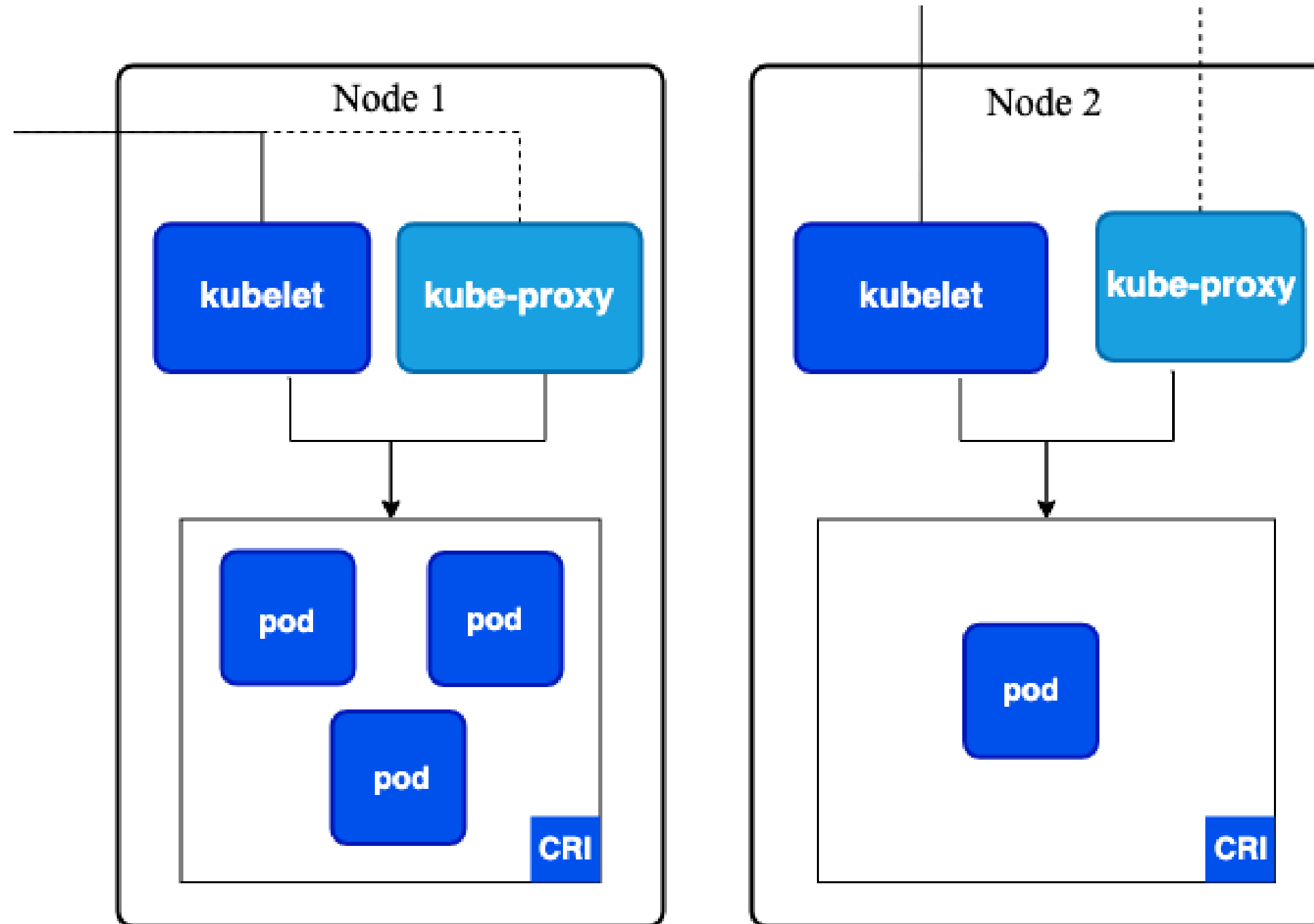
Importance

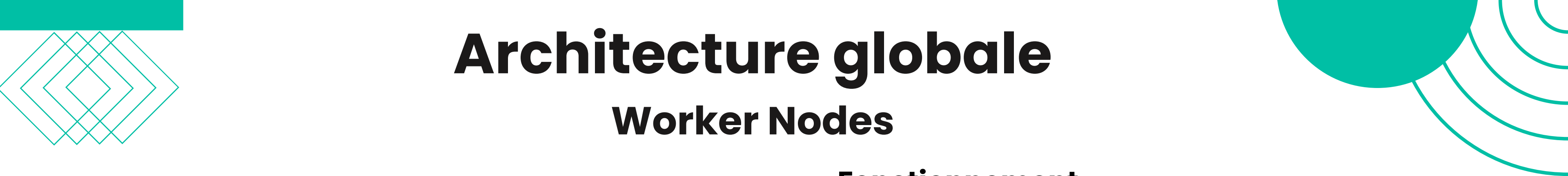
- etcd est critique pour Kubernetes.
- Si etcd est perdu :
 - **le cluster ne peut plus fonctionner correctement.**



Architecture globale

Worker Nodes





Architecture globale

Worker Nodes

Rôle des Worker Nodes

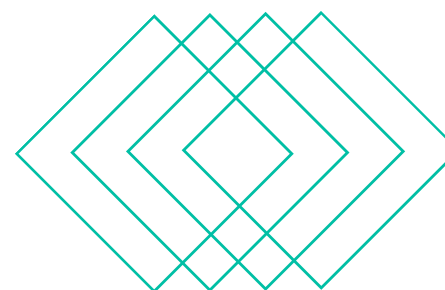
- Les Worker Nodes sont les machines qui exécutent réellement les applications.
- Ils hébergent les Pods et les conteneurs.

Responsabilité

- exécutent les conteneurs
- communiquent avec le Control Plane
- rapportent leur état

Fonctionnement

- Chaque Node peut exécuter plusieurs Pods :
 - Node 1
 - |— Pod
 - |— Pod
 - |— Pod
 - Node 2
 - |— Pod
 - |— Pod





Architecture globale

Worker Nodes (kubelet, kube-proxy et container runtime)

Chaque Worker Node possède plusieurs composants.

kubelet

Le kubelet est l'agent Kubernetes du Node.

- reçoit les instructions du Control Plane
- lance les Pods
- surveille les conteneurs

kube-proxy

Le kube-proxy gère le réseau des Pods.

Il permet :

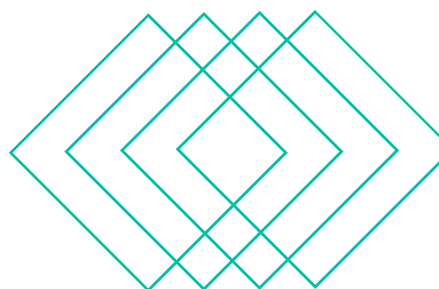
- la communication entre Pods
- le routage du trafic vers les services

Container Runtime

Le container runtime exécute les conteneurs.

Exemples :

- containerd (par default)
- Docker (anciennement)
- CRI-O





Les objets Kubernetes

Dans Kubernetes, tout est représenté par des objets (resources).

- Un objet décrit :
 - l'état souhaité du système
 - les ressources nécessaires
 - le comportement de l'application

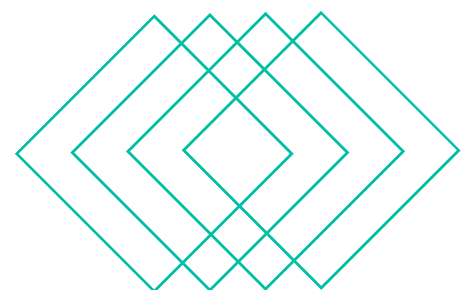
Exemple d'objets Kubernetes

- Pod
- ReplicaSet
- Deployment
- Service
- ConfigMap
- Secret
- Namespace

Principe important :

- Les objets sont généralement décrits dans des fichiers YAML.
- Exemple :

```
kind: Pod  
metadata:  
  name: nginx
```



Déclaration d'objets avec YAML

- Kubernetes fonctionne avec une approche déclarative.
- On décrit l'état souhaité du système dans un fichier YAML.

Exemple simple

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
```

```
spec:
  containers:
  - name: nginx
    image: nginx
```

Déploiement

```
kubectl apply -f pod.yaml
```

- Kubernetes :
 - lit la configuration
 - crée les ressources nécessaires
 - maintient cet état



Le Pod (concept central)

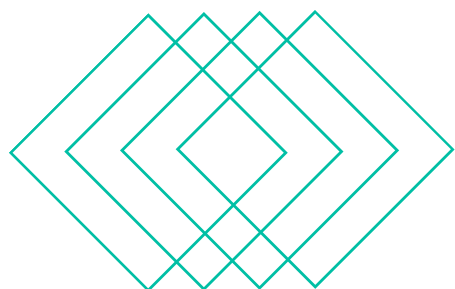
- Le Pod est l'unité de base dans Kubernetes.
- C'est le plus petit objet déployable.

Un Pod contient

- un ou plusieurs conteneurs
- une adresse IP
- un espace réseau partagé
- éventuellement un stockage partagé

Exemple

- Pod
 - |—— container API
 - |—— container logger
- Les conteneurs dans un Pod :
 - partagent le même réseau
 - communiquent via localhost



Exemple de Pod

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
```

```
spec:
  containers:
  - name: nginx
    image: nginx
    ports:
    - containerPort: 80
```

- **Création du Pod**
 - `kubectl apply -f pod.yaml`
- **Vérifier**
 - `kubectl get pods`

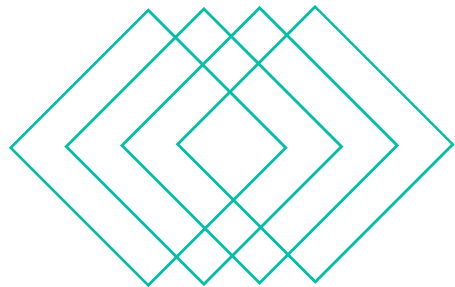


Cycle de vie d'un Pod

Un Pod peut passer par plusieurs états.

États principaux

Etat	Description
Pending	Pod en création
Running	Pod actif
Succeeded	terminé correctement
Failed	erreur
Unknown	état inconnu



Cycle de vie d'un Pod

Un Pod peut passer par plusieurs états.

États principaux

Etat	Description
Pending	Pod en création
Running	Pod actif
Succeeded	terminé correctement
Failed	erreur
Unknown	état inconnu

Important

- Les Pods sont éphémères.
 - Si un Pod disparaît :
 - Kubernetes peut en recréer un.



Labels et Selectors

- Les labels sont des métadonnées attachées aux objets.
- Ils servent à identifier et organiser les ressources.

Exemple

```
metadata:  
  labels:  
    app: nginx  
    env: production
```

Exemple de selector

```
selector:  
  app: nginx
```

Utilisation

- Les labels permettent :
 - sélectionner des Pods
 - regrouper des ressources
 - connecter des services



ReplicaSet

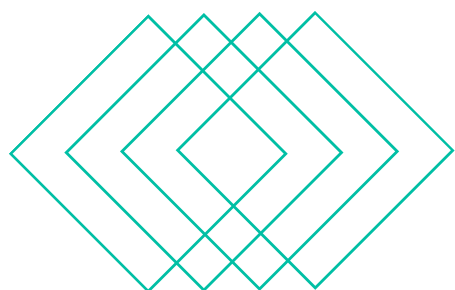
Un ReplicaSet garantit qu'un certain nombre de Pods sont toujours en cours d'exécution.

Exemple

- On demande :
 - 3 Pods nginx
 - Si un Pod tombe :
 - ReplicaSet en crée un nouveau.

Principe

- Etat souhaité : 3 Pods
- Etat actuel : 2 Pods
- ReplicaSet → crée 1 Pod



Exemple ReplicaSet

```
apiVersion: apps/v1          # Version de l'API Kubernetes utilisée (objets applicatifs)
kind: ReplicaSet              # Type d'objet : ici un ReplicaSet

spec:                          # Spécification de l'état souhaité
  replicas: 3                  # Nombre de Pods souhaités en permanence

  selector:                    # Permet de sélectionner les Pods gérés par ce ReplicaSet
    matchLabels:                # Sélection basée sur des labels
      app: nginx                 # Le ReplicaSet gère les Pods ayant le label app=nginx

  template:                     # Modèle utilisé pour créer les Pods
    metadata:
      labels:
        app: nginx              # Label attribué aux Pods créés
                                #Doit correspondre au selector !

    spec:                       # Spécification du Pod
      containers:                # Liste des conteneurs dans le Pod
        - name: nginx            # Nom du conteneur
          image: nginx            # Image Docker utilisée (ici nginx)
```



Deployment

- Dans la pratique, on utilise rarement directement les ReplicaSets.
- On utilise plutôt des Deployments.

Pourquoi ?

- Le Deployment permet :
- gérer les ReplicaSets
- faire des mises à jour
- gérer les rollbacks

Le Deployment est l'objet le plus utilisé
Pour déployer une application.

Exemple Deployment

apiVersion: apps/v1
kind: Deployment

Version de l'API Kubernetes pour les applications
Type d'objet : ici un Deployment

metadata:

name: nginx-deployment # Nom du Deployment (identifiant dans le cluster)

spec:

Description de l'état souhaité

replicas: 3

Nombre de Pods souhaités en permanence

selector:

Permet de sélectionner les Pods gérés par ce Deployment

matchLabels:

Sélection basée sur les labels

app: nginx

Le Deployment gère les Pods ayant le label app=nginx

template:

Modèle utilisé pour créer les Pods

metadata:

labels:

app: nginx

Label attribué aux Pods créés

Doit correspondre au selector !

spec:

Spécification du Pod

containers:

Liste des conteneurs dans le Pod

- name: nginx

Nom du conteneur

image: nginx

Image utilisée (ici nginx)



Scaling

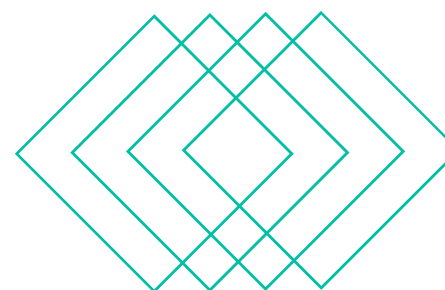
Kubernetes permet de changer facilement le nombre de Pods.

Exemple

```
kubectl scale deployment nginx-deployment --replicas=5
```

Résultat

- Avant :
 - 3 Pods
- Après :
 - 5 Pods
- Kubernetes crée automatiquement les nouveaux Pods.





Rolling Update

Kubernetes permet de mettre à jour une application sans interruption.

Exemple

Changer l'image :

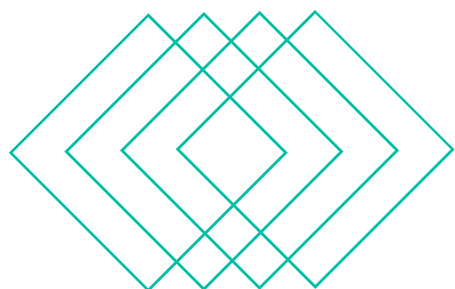
```
kubectl set image deployment/nginx nginx=nginx:1.25
```

Fonctionnement

- Kubernetes :
 - crée de nouveaux Pods
 - supprime progressivement les anciens

Avantage

- pas d'interruption de service
- déploiement progressif
- possibilité de rollback





Namespace

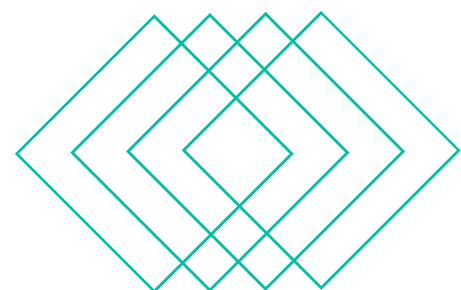
Les Namespaces permettent d'organiser les ressources dans un cluster.

Exemple

- On peut avoir :
 - namespace dev
 - namespace test
 - namespace production
- Chaque namespace possède :
 - ses Pods
 - ses services
 - ses deployments

Création

- `kubectl create namespace dev`





kubectl

kubectl est la ligne de commande pour interagir avec Kubernetes.

Rôle

- Il permet de :
 - créer des ressources
 - consulter l'état du cluster
 - modifier des objets
 - supprimer des ressources

Exemple

```
kubectl get pods
```

Toutes les actions passent par l'API Server.



Les commandes de base



Voir les ressources

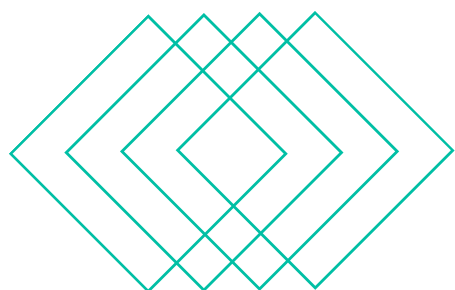
- `kubectl get pods`
- `kubectl get nodes`
- `kubectl get deployments`

Format étendu

- `kubectl get pods -o wide`

Namespace

`kubectl get pods -n dev`



Les commandes de base

Voir les ressources

- `kubectl get pods`
- `kubectl get nodes`
- `kubectl get deployments`

Format étendu

- `kubectl get pods -o wide`

Namespace

`kubectl get pods -n dev`

Créer et supprimer des ressources

- Créer
 - `kubectl apply -f fichier.yaml`
- Supprimer
 - `kubectl delete -f fichier.yaml`
 - `kubectl delete pod nginx`
- `apply` → créer ou mettre à jour
- `delete` → supprimer

Les commandes de base

Inspecter les ressources

- Détails complets
 - `kubectl describe pod nginx`
- Logs
 - `kubectl logs nginx`
- voir pourquoi un Pod ne démarre pas
- vérifier une erreur applicative

Exécuter des commandes dans un Pod

- Accéder à un conteneur
 - `kubectl exec -it nginx -- bash`
- Utilisation
 - tester une API
 - vérifier un fichier
 - debug réseau

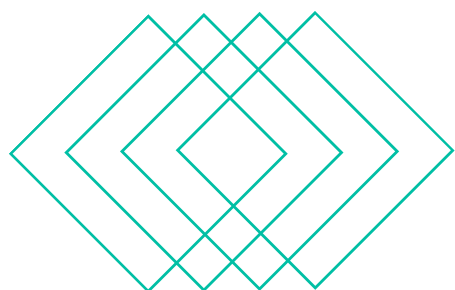


Les commandes de base

Gestion des Deployments

- Voir les deployments
 - `kubectl get deployments`
- Scaling
 - `kubectl scale deployment nginx --replicas=5`
- Mise à jour
 - `kubectl set image deployment/nginx nginx=nginx:1.25`

Kubernetes gère automatiquement les Pods derrière



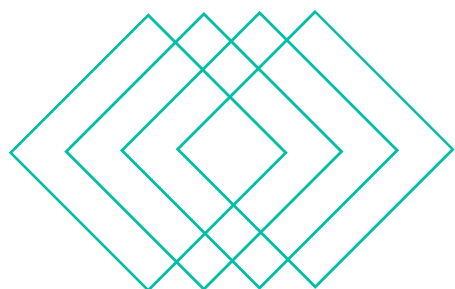


Ce que vous devez encore apprendre

- Ingress → exposer des applications web (HTTP/HTTPS)
- Volumes & stockage persistant → conserver les données
- ConfigMap & Secrets → gérer la configuration
- Autoscaling (HPA) → adapter automatiquement le nombre de Pods
- Sécurité (RBAC) → contrôler les accès

Compétences à développer

- Lire et écrire des fichiers YAML complexes
- Comprendre les erreurs Kubernetes
- Déboguer avec :
 - kubectl describe
 - kubectl logs
- Comprendre l'architecture réelle d'un cluster



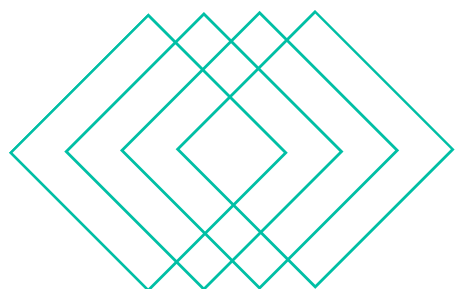


Stockage Kubernetes

Pourquoi le stockage est important?

- Les Pods sont éphémères :
 - supprimés
 - recréés
 - perte des données
- Exemple:
 - Base de données dans un Pod → données perdues

Kubernetes propose des volumes



Stockage Kubernetes

Volume Kubernetes

- Un volume permet de stocker des données :
 - indépendamment du conteneur
 - partageable entre conteneurs
- Exemple (YAML):
 - volumes:
 - – name: task-pv-volume

```

pods/storage/pv-volume.yaml Copy pods/storage/pv-volume.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: task-pv-volume
  labels:
    type: local
spec:
  storageClassName: manual
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/mnt/data"
```

Stockage Kubernetes

Volume Kubernetes

- Un volume permet de stocker des données :
 - indépendamment du conteneur
 - partageable entre conteneurs
- Exemple (YAML):
 - volumes:
 - – name: task-pv-volume

```
 pods/storage/pv-volume.yaml Copy pods/storage/pv-volume.yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: task-pv-volume
  labels:
    type: local
spec:
  storageClassName: manual
  capacity:
    storage: 10Gi
  accessModes:
    - ReadWriteOnce
  hostPath:
    path: "/mnt/data"
```

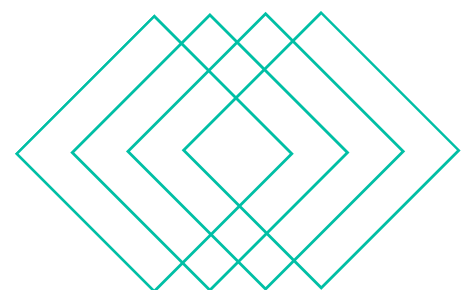


Stockage Kubernetes

Volume Kubernetes

Types de volumes

- Types principaux
 - emptyDir → temporaire
 - hostPath → disque du node
 - local → stockage local optimisé (production)
 - volumes cloud
 - AWS (EBS (Elastic Block Store))
 - GCP (Google Cloud Platform)
 - GCE Persistent Disk
 - Azure → Azure Disk, Azure File
 - Important
 - certains volumes sont persistants, d'autres non





Stockage Kubernetes

Volume Kubernetes

Types de volumes

- Types principaux
 - emptyDir → temporaire
 - hostPath → disque du node
 - local → stockage local optimisé (production)
 - volumes cloud
 - AWS (EBS (Elastic Block Store)
 - GCP (Google Cloud Platform)
 - GCE Persistent Disk
 - Azure → Azure Disk, Azure File
 - Important
 - certains volumes sont persistants, d'autres non
- Volumes réseau
 - NFS → stockage partagé réseau
 - CephFS → distribué et scalable
 - GlusterFS → système distribué
- Volumes spéciaux
 - configMap → fichiers de configuration
 - secret → données sensibles
 - downwardAPI → infos du Pod
 - projected → combinaison de volumes

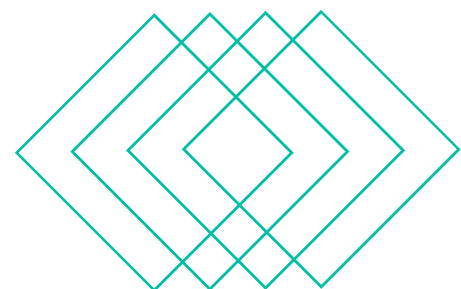


Stockage Kubernetes

Volume Kubernetes

Types de volumes

- emptyDir
 - emptyDir → temporaire
 - Un volume temporaire créé pour un Pod.
 - Il est créé quand le Pod démarre
 - Il est supprimé quand le Pod est supprimé
 - partagé entre conteneurs du même Pod
 - stockage local (disque ou mémoire)
 - durée de vie = celle du Pod
 - YAML:
 - volumes:
 - – name: cache
 - emptyDir: {}



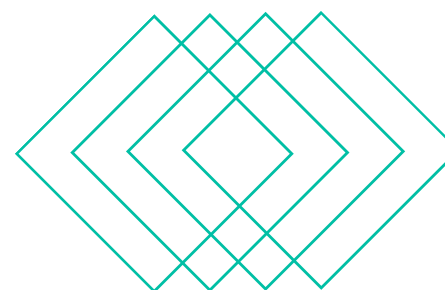


Stockage Kubernetes

Volume Kubernetes

Types de volumes

- hostPath
 - hostPath → disque du node
 - hostPath permet d'utiliser un dossier du système du Node (machine physique).
 - accès direct au disque du Node
 - persistant tant que le Node existe
 - dépend du Node spécifique
 - Cas d'usage
 - logs système
 - monitoring
 - accès à des fichiers locaux
- Limites importantes
 - pas portable (lié à un Node)
 - problème si Pod change de Node
 - risque de sécurité (accès au système)
- Exemple=
 - volumes:
 - – name: host-volume
 - hostPath:
 - path: /data

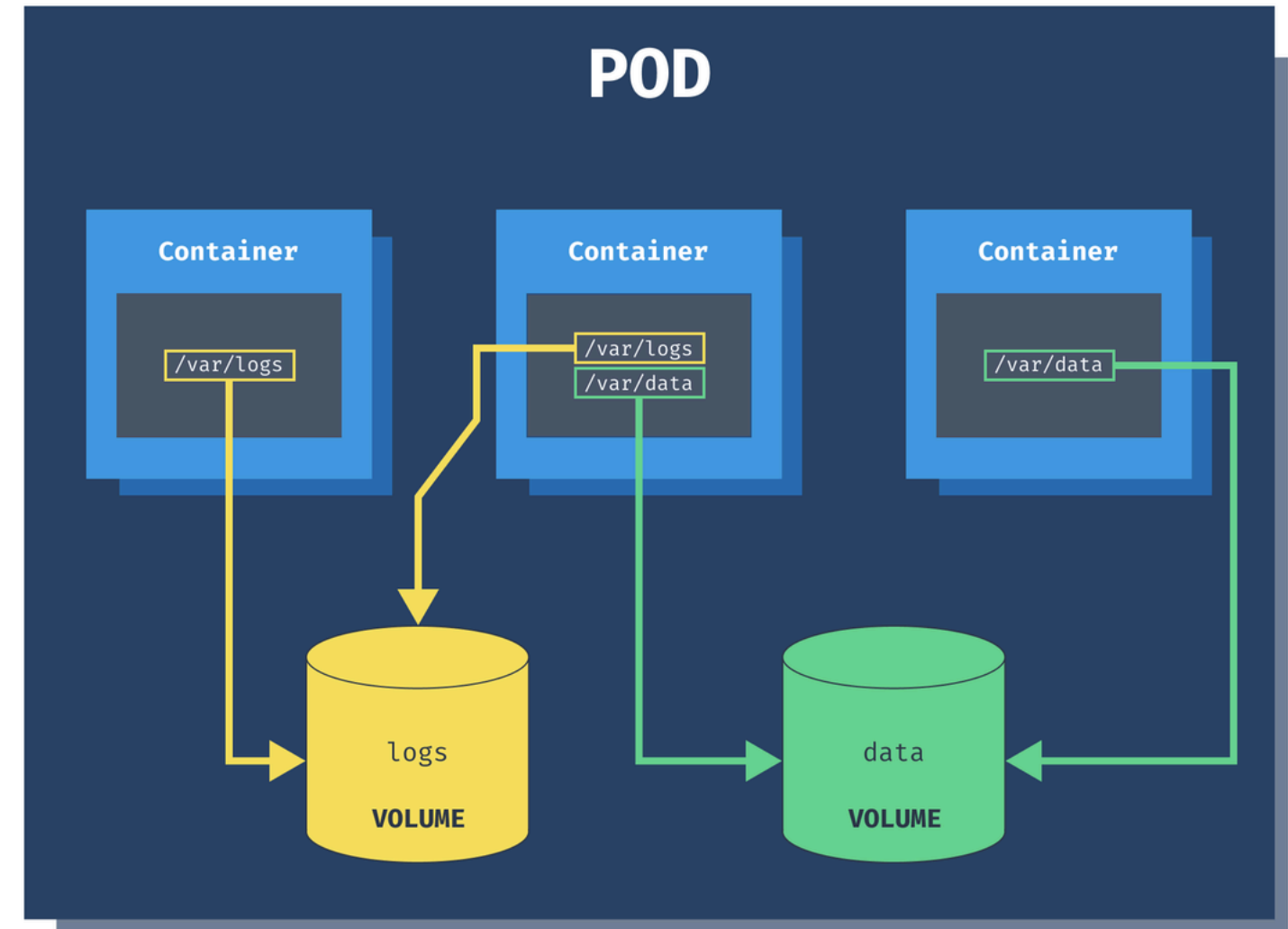


Stockage Kubernetes

Volume Kubernetes

Persistent Volume

- Un Persistent Volume (PV) est un espace de stockage :
 - indépendant des Pods
 - géré par le cluster



Stockage Kubernetes

Volume Kubernetes

Persistent Volume Claim (PVC)

- Un PVC est une demande de stockage.
- Fonctionnement
 - Pod → PVC → PV
- abstraction du stockage
- demander du stockage sans connaître l'infrastructure
- Fonctionnement global:
 - Pod → PVC → PV → Disque réel
 - i. le Pod utilise un PVC
 - ii. le PVC demande du stockage
 - iii. Kubernetes associe un PV compatible

```
kind: PersistentVolumeClaim
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```


Stockage Kubernetes

Volume Kubernetes

Relation PV / PVC

<u>PV</u>	<u>PVC</u>
stockage réel	demande de stockage
créé par admin / cloud	créé par développeur
ressource physique	abstraction
indépendant des Pods	utilisé par les Pods

```
kind: PersistentVolumeClaim
spec:
  resources:
    requests:
      storage: 1Gi
```

- Dans le Pod

```
volumes:
  - name: data
    persistentVolumeClaim:
      claimName: my-pvc
```



Stockage Kubernetes

Volume Kubernetes

Comment créer un Persistent Volume (PV)

```
apiVersion: v1
kind: PersistentVolume

metadata:
  name: my-pv

spec:
  capacity:
    storage: 1Gi

  accessModes:
    - ReadWriteOnce

  hostPath:
    path: /mnt/data
```

Stockage Kubernetes

Volume Kubernetes

Comment créer un Persistent Volume (PV)

```
apiVersion: v1
kind: PersistentVolume
```

```
metadata:
  name: my-pv
```

```
spec:
  capacity:
    storage: 1Gi
```

```
accessModes:
  - ReadWriteOnce
```

```
hostPath:
  path: /mnt/data
```

- Appliquer le PV
 - `kubectl apply -f pv.yaml`
- Vérifier
 - `kubectl get pv`



Stockage Kubernetes

Volume Kubernetes

Comment créer un Persistent Volume (PV)

- Appliquer le PV
 - `kubectl apply -f pv.yaml`
- Vérifier
 - `kubectl get pv`

Modes d'accès

Mode	Description
ReadWriteOnce	un seul node
ReadOnlyMany	lecture seule multiple
ReadWriteMany	multi accès

```
apiVersion: v1
kind: PersistentVolume

metadata:
  name: my-pv

spec:
  capacity:
    storage: 1Gi

  accessModes:
    - ReadWriteOnce

  hostPath:
    path: /mnt/data
```

Stockage Kubernetes

Volume Kubernetes

Créer un PVC pour utiliser le PV

```
apiVersion: v1
kind: PersistentVolumeClaim

metadata:
  name: my-pvc

spec:
  accessModes:
    - ReadWriteOnce

  resources:
    requests:
      storage: 1Gi
```

- Appliquer le PVC
 - `kubectl apply -f pvc.yaml`
- Vérifier
 - `kubectl get pv`
 - `kubectl get pvc`

Utiliser le PVC dans un Pod

```
volumes:
  - name: storage
    persistentVolumeClaim:
      claimName: my-pvc
```



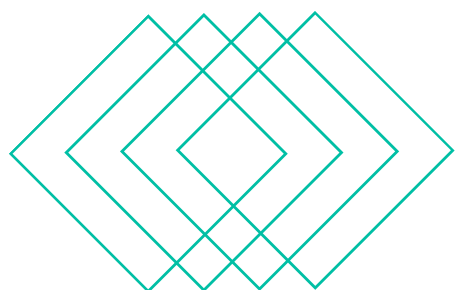
ConfigMap & Secret

Problème

- Dans une application, on a souvent :
 - variables d'environnement
 - fichiers de configuration
 - mots de passe / clés API

Mauvaise pratique

- Mettre ces informations directement dans le code :
 - `DB_PASSWORD = "123456"`
 - difficile à modifier
 - dangereux (sécurité)
 - non portable





ConfigMap & Secret

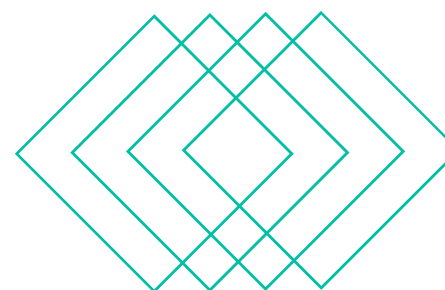
Problème

- Dans une application, on a souvent :
 - variables d'environnement
 - fichiers de configuration
 - mots de passe / clés API

Mauvaise pratique

- Mettre ces informations directement dans le code :
 - DB_PASSWORD = "123456"
 - difficile à modifier
 - dangereux (sécurité)
 - non portable

- **Kubernetes sépare :**
 - **Application $\neq$ Configuration**
- **Deux outils**
 - **ConfigMap $\rightarrow$ données non sensibles**
 - **Secret $\rightarrow$ données sensibles**



ConfigMap & Secret

ConfigMap

- Un ConfigMap stocke des données de configuration :
 - variables d'environnement
 - fichiers texte
 - paramètres d'application

Création du ConfigMap

```
apiVersion: v1
kind: ConfigMap

metadata:
  name: app-config

data:
  ENV: production
  DEBUG: "false"
  APP_NAME: "mon-app"
```


ConfigMap & Secret

Utilisation dans un Pod

```
apiVersion: v1
kind: Pod

metadata:
  name: my-app

spec:
  containers:
  - name: app
    image: nginx

    env:
      - name: ENV
        valueFrom:
          configMapKeyRef:
            name: app-config
            key: ENV

      - name: DEBUG
        valueFrom:
          configMapKeyRef:
            name: app-config
            key: DEBUG
```

Création du ConfigMap

```
apiVersion: v1
kind: ConfigMap

metadata:
  name: app-config

data:
  ENV: production
  DEBUG: "false"
  APP_NAME: "mon-app"
```

ConfigMap & Secret

Utilisation dans un Pod

```
apiVersion: v1
kind: Pod

metadata:
  name: my-app

spec:
  containers:
  - name: app
    image: nginx

    env:
      - name: ENV
        valueFrom:
          configMapKeyRef:
            name: app-config
            key: ENV

      - name: DEBUG
        valueFrom:
          configMapKeyRef:
            name: app-config
            key: DEBUG
```

Création du ConfigMap

```
apiVersion: v1
kind: ConfigMap

metadata:
  name: app-config

data:
  ENV: production
  DEBUG: "false"
  APP_NAME: "mon-app"
```

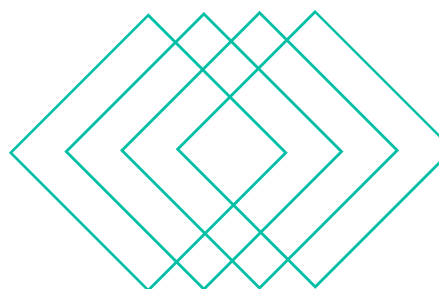
Le ConfigMap permet de changer la configuration sans modifier l'image Docker



ConfigMap & Secret

Secret

- Un Secret permet de stocker des données sensibles :
 - mots de passe
 - clés API
 - tokens
- Les données sont encodées en base64 (pas chiffrées).



ConfigMap & Secret

Secret

- Un Secret permet de stocker des données sensibles :
 - mots de passe
 - clés API
 - tokens
- Les données sont encodées en base64 (pas chiffrées).

Créer le Secret

```
apiVersion: v1
kind: Secret

metadata:
  name: app-secret

type: Opaque

data:
  password: bXlwYXNzd29yZA==
```

ConfigMap & Secret

Secret

- Un Secret permet de stocker des données sensibles :
 - mots de passe
 - clés API
 - tokens
- Les données sont encodées en base64 (pas chiffrées).

Créer le Secret

```
apiVersion: v1
kind: Secret

metadata:
  name: app-secret

type: Opaque

data:
  password: bXlwYXNzd29yZA==
```

ConfigMap & Secret

Secret

- Un Secret permet de stocker des données sensibles :
 - mots de passe
 - clés API
 - tokens
- Les données sont encodées en base64 (pas chiffrées).

Encoder les données
echo -n "mypassword" | base64

Créer le Secret

```
apiVersion: v1
kind: Secret

metadata:
  name: app-secret

type: Opaque

data:
  password: bXlwYXNzd29yZA==
```

ConfigMap & Secret

Utilisation d'un Secret

Injection dans un Pod

```
apiVersion: v1
kind: Pod

metadata:
  name: my-app

spec:
  containers:
  - name: app
    image: nginx

    env:
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: app-secret
          key: password
```

Dans le conteneur

echo \$DB_PASSWORD

mypassword

ConfigMap & Secret

Utilisation d'un Secret

Injection dans un Pod

```
apiVersion: v1
kind: Pod

metadata:
  name: my-app

spec:
  containers:
  - name: app
    image: nginx

    env:
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: app-secret
          key: password
```

Alternative: utiliser comme fichier

```
volumes:
  - name: secret-volume
    secret:
      secretName: app-secret
```

Dans le conteneur
echo \$DB_PASSWORD
mypassword

ConfigMap & Secret

Utilisation d'un Secret

Injection dans un Pod

```
apiVersion: v1
kind: Pod

metadata:
  name: my-app

spec:
  containers:
  - name: app
    image: nginx

    env:
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: app-secret
          key: password
```

Alternative: utiliser comme fichier

```
volumes:
  - name: secret-volume
    secret:
      secretName: app-secret
```

Dans le conteneur
echo \$DB_PASSWORD
mypassword

Le Secret sera disponible sous forme de fichier dans le conteneur.

Différence ConfigMap vs Secret

Critère	ConfigMap	Secret
Type de données	non sensibles	sensibles
Encodage	non	base64
Usage	config	sécurité



Service

Problème

- Les Pods :
 - ont une IP dynamique
 - peuvent être recréés
 - changent d'adresse
 - Example:
 - Pod A → 10.0.0.1
 - (restart)
 - Pod A → 10.0.0.8
 - Solution:
 - utiliser un Service
- 

Service

Problème

- Les Pods :
 - ont une IP dynamique
 - peuvent être recréés
 - changent d'adresse
- Example:
 - Pod A → 10.0.0.1
 - (restart)
 - Pod A → 10.0.0.8
- Solution:
 - utiliser un Service

Qu'est-ce qu'un Service ?

- Un Service est une abstraction qui :
 - fournit une IP stable
 - permet d'accéder à un groupe de Pods
 - Client → Service → Pods
- **Le Service :**
 - **utilise des labels**
 - **répartit le trafic (load balancing)**
 - **accès stable même si les Pods changent**

Service

Types de Services

Type	Accès
ClusterIP	interne au cluster
NodePort	via IP du node + port
LoadBalancer	via cloud

Exemple

- **ClusterIP**
 - **Service accessible uniquement entre Pods**
- **NodePort**
 - **`http://IP_NODE:30001`**
- **LoadBalancer**
 - **`http://external-ip`**

Ce qu'on veut
`example.com/app`
`example.com/api`

Exemple Service (NodePort)

```
apiVersion: v1
kind: Service

metadata:
  name: my-service

spec:
  type: NodePort

  selector:
    app: my-app

  ports:
    - port: 80
      targetPort: 80
      nodePort: 30001
```



Ingress

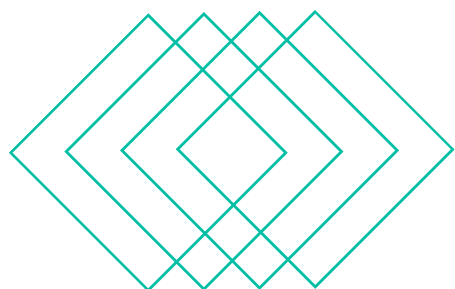
Problème

- Les Services Kubernetes (NodePort / LoadBalancer) :
 - exposent un seul service
 - utilisent des ports différents
 - ne gèrent pas les URLs

Qu'est-ce qu'un Ingress

- Un Ingress est une ressource Kubernetes qui :
 - gère l'accès HTTP/HTTPS
 - redirige le trafic vers les Services
 - agir comme un routeur HTTP intelligent
- 

Internet → Ingress → Service → Pods



Ingress

Cas d'usage
/app → service-app
/api → service-api

un seul point d'entrée pour plusieurs services

```
apiVersion: networking.k8s.io/v1
kind: Ingress

metadata:
  name: my-ingress

spec:
  rules:
    - http:
        paths:
          - path: /app
            pathType: Prefix
            backend:
              service:
                name: service-app
                port:
                  number: 80
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: service-api
                port:
                  number: 80
```



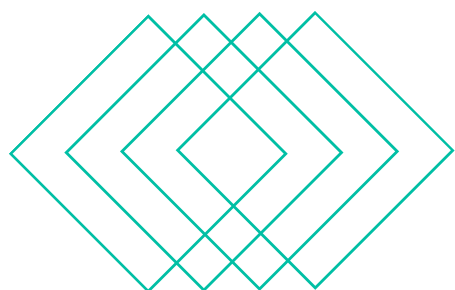

Ingress Controller

Problème

- Un Ingress ne fonctionne pas tout seul.
 - Il faut un Ingress Controller
 - Exemples:
 - NGINX Ingress
 - Traefik
 - HAProxy

Rôle

- Le controller :
 - lit les règles Ingress
 - configure le routage
 - reçoit le trafic
- Ingress = règles
- Controller = moteur



Exemple Ingress + Controller

```
apiVersion: networking.k8s.io/v1    # Version de l'API Kubernetes pour Ingress

kind: Ingress                        # Type de ressource : Ingress

metadata:
  name: my-ingress                  # Nom de l'Ingress

spec:
  rules:                            # Règles de routage

    - host: example.com             # Nom de domaine utilisé

      http:                          # Liste des chemins (URLs)
        paths:
          - path: /app               # URL : example.com/app
            pathType: Prefix         # Correspond à tous les chemins commençant par /app

        backend:
          service:
            name: my-service         # Nom du Service cible
            port:
              number: 80             # Port du Service
```

Exemple Ingress + Controller

```
apiVersion: networking.k8s.io/v1  # Version de l'API Kubernetes pour Ingress

kind: Ingress                      # Type de ressource : Ingress

metadata:
  name: my-ingress                # Nom de l'Ingress

spec:
  rules:                          # Règles de routage

    - host: example.com           # Nom de domaine utilisé

      http:
        paths:                    # Liste des chemins (URLs)

          - path: /app             # URL : example.com/app
            pathType: Prefix       # Correspond à tous les chemins commençant par /app

          backend:
            service:
              name: my-service     # Nom du Service cible
              port:
                number: 80         # Port du Service
```

Exemple installation (NGINX)

```
kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/cloud/deploy.yaml
```

User → Ingress → Service → Pods



Autoscaling

Problème

- Le trafic d'une application varie :
 - pics (beaucoup d'utilisateurs)
 - creux (peu d'activité)
 - Sans autoscaling
 - trop de Pods → gaspillage de ressources
 - pas assez de Pods → application lente
 - Objectif
 - adapter automatiquement le nombre de Pods
- 



Autoscaling

Problème

- Le trafic d'une application varie :
 - pics (beaucoup d'utilisateurs)
 - creux (peu d'activité)
 - Sans autoscaling
 - trop de Pods → gaspillage de ressources
 - pas assez de Pods → application lente
 - Objectif
 - adapter automatiquement le nombre de Pods
- 

Horizontal Pod Autoscaler (HPA)

- Le Horizontal Pod Autoscaler (HPA) ajuste automatiquement :
 - le nombre de Pods d'un Deployment
 - Charge ↑ → + Pods
 - Charge ↓ → - Pods
 - Basé sur
 - **CPU**
 - **mémoire**
 - **métriques personnalisées**
- 

Autoscaling

Principe de fonctionnement

- Kubernetes surveille le CPU
- compare avec un seuil
- ajuste le nombre de Pods
- Exemple:
 - CPU > 70% → scale up
 - CPU < 30% → scale down
- Important
 - fonctionne sur un Deployment, pas directement sur un Pod

Exemple de commande HPA

- `kubectl autoscale deployment nginx --cpu-percent=50 --min=1 --max=10`
- **--cpu-percent=50 → cible CPU**
- **--min → minimum de Pods**
- **--max → maximum**

Exemple YAML HPA

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler

metadata:
  name: my-hpa

spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-app

  minReplicas: 1
  maxReplicas: 10

  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

Autoscaling

Problème

- Avec HPA, on augmente le nombre de Pods
- Mais parfois le problème est ailleurs :
 - Pod trop petit (manque CPU / RAM)
 - mauvaise configuration des ressources
- Exemple:
 - Pod limité à 100m CPU → saturé

Vertical Pod Autoscaler (VPA)

- Le Vertical Pod Autoscaler (VPA) ajuste automatiquement :
 - les ressources d'un Pod
 - CPU
 - mémoire
- Charge ↑ → + CPU / RAM
- Charge ↓ → - CPU / RAM
- **Exemple**
 - **HPA → 3 Pods → 6 Pods**
 - **VPA → 1 Pod → plus puissant**
 -

Exemple YAML VPA

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler

metadata:
  name: my-vpa

spec:
  targetRef:
    apiVersion: "apps/v1"
    kind: Deployment
    name: my-app

  updatePolicy:
    updateMode: "Auto"
```

Exemple YAML VPA

```
apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler

metadata:
  name: my-vpa

spec:
  targetRef:
    apiVersion: "apps/v1"
    kind: Deployment
    name: my-app

  updatePolicy:
    updateMode: "Auto"
```

```
apiVersion: autoscaling.k8s.io/v1 # Version de l'API pour le Vertical Pod Autoscaler

kind: VerticalPodAutoscaler # Type de ressource : VPA

metadata:
  name: my-vpa # Nom du VPA

spec:
  targetRef: # Cible du VPA (objet à autoscaler)
    apiVersion: apps/v1 # Version de l'objet cible
    kind: Deployment # Type de ressource (Deployment)
    name: my-app # Nom du Deployment ciblé

  updatePolicy:
    updateMode: "Auto" # Mode automatique (modifie les ressources automatiquement)

  resourcePolicy: # Politique de gestion des ressources
    containerPolicies: # Liste des règles par conteneur

    - containerName: "*" # "*" = s'applique à tous les conteneurs du Pod

      minAllowed: # Limite minimale autorisée
        cpu: 100m # CPU minimum (100 milliCPU)
        memory: 128Mi # Mémoire minimum

      maxAllowed: # Limite maximale autorisée
        cpu: 1 # CPU max = 1 core
        memory: 1Gi # Mémoire max
```